https: / / github.com / fhlsteven / CLR_via_CSharp / blob / main / chapters / ch13_Interfaces.md 我现在在学习《CLR via C#》这本书的第13章，内容如链接所示，我们后面将围绕这个讨论

用户:

https://github.com/fhlsteven/CLR_via_CSharp/blob/main/chapters/ch13_Interfaces.md

我现在在学习《CLR via C#》这本书的第13章，内容如链接所示，我们后面将围绕这个讨论，我也会向你请教。

元宝:

好的，第13章的接口内容我已经看过了，脉络很清晰——从 CLR 不支持多继承、接口作为"缩水版多继承"切入，讲到接口定义、类实现接口时 C#/CLR 对方法签名的处理（`virtual` / `sealed`、EIMI 显式接口实现）、泛型接口带来的类型安全与减少装箱、接口约束、再到"基类 vs 接口"的选型原则（IS-A vs CAN-DO、版本控制、FCL 里 Stream 用继承 / 集合用接口的对仗案例）。

这一章几个容易踩坑、也常被问到的点我先帮你标一下，后面你问到哪块我们再深聊：

- **EIMI（显式接口方法实现）的双面性**：私有 + non-virtual，防直接调用但也导致派生类调不到、文档和 IntelliSense 不友好、值类型转接口必装箱。书里给的解法是"基类里同时留一个 protected/public virtual，让派生类 override"。
- **泛型接口的三个红利**：编译时类型安全、值类型少装箱、同一类可对同一接口用不同 T 实现多次（如 `IComparable<int>` + `IComparable<string>`）。
- **接口约束** `where T : IComparable<T>` 能让值类型调接口方法不装箱，这是非约束写法做不到的，IL 层面有特殊指令。
- **基类 vs 接口**那张"决策表"——易用性、一致性、版本控制（加成员接口破坏性变更 vs 基类可继承），FCL 里 Stream（IS-A 用继承）vs 集合（CAN-DO 用接口）对照着看最直观。

随时抛问题，不管是概念澄清、代码例子、还是你做题/写代码时卡住的细节，咱们顺着这章聊。

用户:

这一部分例子不太懂：

public interface IDisposable {

```
    void Dispose();
```

}

public interface IEnumerable {

```
    IEnumerator GetEnumerator();
```

}

public interface IEnumerable<T> : IEnumerable {

```
    new IEnumerator<T> GetEnumerator();
```

}

public interface ICollection<T> : IEnumerable<T>, IEnumerable {

```
    void     Add(T item);

    void     Clear();

    Boolean  Contains(T item);

    void     CopyTo(T[] array, Int32 arrayIndex);

    Boolean  Remove(T item);

    Int32    Count      { get; }  // 只读属性

    Boolean  IsReadOnly { get; }  //只读属性
```

}

元宝:

这几个接口其实是 .NET 集合体系的“骨架”，你看着晕主要是因为：**接口继承 + `new` 关键字 + 属性本质**这三件事叠在一起了。我拆开讲，你对着代码看会非常清楚。

1️⃣ 先看 `IDisposable` —— 接口里只有“契约”

```
    public interface IDisposable {
        void Dispose();
    }
```

这是一个**最朴素的接口**：

- 接口里**不能有实现**
- 只有方法签名
- 实现这个接口的类，**必须提供 `Dispose()` 的具体实现**

👉 它的作用就是一句话：

> “凡是实现了我的类型，都承诺自己知道怎么‘释放资源’。”

2️⃣ `IEnumerable`：非泛型时代的“老接口”

```
public interface IEnumerable {
    IEnumerator GetEnumerator();
}
```

它的作用是：**支持 foreach。**

```
foreach (var x in obj) { }
```

编译器实际做的是：

```
IEnumerator e = obj.GetEnumerator();
while (e.MoveNext()) { }
```

⚠️ 注意：

- `IEnumerator` 返回的是 **非泛型枚举器**
- 这意味着：
  - 返回值是 `object`
  - 值类型会 **装箱**
  - 没有编译期类型安全

这是 .NET 1.0 的历史遗留问题。

3️⃣ `IEnumerable<T>`：泛型版本的“升级接口”

```
public interface IEnumerable<T> : IEnumerable {
    new IEnumerator<T> GetEnumerator();
}
```

这里有两个关键点：

✅ ① 它继承了 `IEnumerable`

这意味着：

> **只要实现了 `IEnumerable<T>`，就自动“也是” `IEnumerable`**

这是为了**向后兼容**：

- 老代码只认识 `IEnumerable`
- 新代码可以用 `IEnumerable<T>`
- 同一个对象，两种都能用

✅ ② `new IEnumerator<T> GetEnumerator();`

这是最容易困惑的地方。

为什么要 `new` ？

因为父类 `IEnumerable` 里已经有一个：

```
    IEnumerator GetEnumerator();
```

而子类想提供一个**更具体、更安全的版本**：

```
    IEnumerator<T> GetEnumerator();
```

如果不用 `new`，C# 会报警：

> 隐藏继承的成员

用 `new` 的意思是：

> “我知道我在隐藏父接口的方法，这是我有意为之。”

运行时实际发生了什么？

CLR 层面其实会有 **两个 GetEnumerator**：

接口	方法
`IEnumerable`	`GetEnumerator() : IEnumerator`
`IEnumerable<T>`	`GetEnumerator() : IEnumerator<T>`

实现类型通常这样写：

```
    class MyList<T> : IEnumerable<T>
    {
        public IEnumerator<T> GetEnumerator() { ... }

        IEnumerator IEnumerable.GetEnumerator()
            => GetEnumerator(); // 显式接口实现
    }
```

✅ 好处：

- `foreach (T x in list)` → 调用泛型版本（无装箱）
- 老代码 → 调用非泛型版本

4️⃣ `ICollection<T>`：集合能力的“标准契约”

```
public interface ICollection<T> : IEnumerable<T>, IEnumerable {
    void     Add(T item);
    void     Clear();
    Boolean  Contains(T item);
    void     CopyTo(T[] array, Int32 arrayIndex);
    Boolean  Remove(T item);
    Int32    Count      { get; }
    Boolean  IsReadOnly { get; }
}
```

✅ 继承关系解读

```
IEnumerable
   ↑
IEnumerable<T>
   ↑
ICollection<T>
```

所以一个 `ICollection<T>`：

- ✅ 能被 `foreach`
- ✅ 能 `Add / Remove`
- ✅ 知道元素数量
- ✅ 知道自己是否只读

✅ 为什么它还要再写一次 `IEnumerable`？

这是个**历史 + 可读性**问题。

```
: IEnumerable<T>, IEnumerable
```

其实 `IEnumerable<T>` **已经继承了** `IEnumerable`，理论上写不写都一样。

但 FCL 里仍然显式写出来，是为了：

- 让接口“自解释”
- 防止将来接口继承链变化导致语义不清
- 方便工具/文档生成

👉 这一点书里没细说，但面试常考 😄

5️⃣ 关于“只读属性”的本质（非常重要）

```
Int32 Count { get; }
Boolean IsReadOnly { get; }
```

很多人误以为这是“字段”。

❌ 错。

接口里的属性，本质是 **方法对**

```
int get_Count();
bool get_IsReadOnly();
```

接口里：

- **不能声明字段**
- `{ get; }` 只是说：

> “实现类必须提供一个 getter”

实现类可以这样：

```
public int Count
{
    get { return _count; }
}
```

但不能：

```
public int Count; // ❌ 接口不允许
```

6️⃣ 把这些接口串起来的一句话总结

> **`IDisposable` 管资源，`IEnumerable` 管遍历，`IEnumerable<T>` 管类型安全的遍历，`ICollection<T>` 管基本集合操作。**

它们不是“随便设计的”，而是：

- ✅ 向后兼容
- ✅ 类型安全
- ✅ 面向契约
- ✅ 支持多态与泛型优化（减少装箱）

如果你愿意，下一步我可以帮你画一张 **接口继承关系图**，或者结合 `List<T>` / `Array` / `Dictionary<TKey,TValue>` 讲它们在 CLR 层面是怎么实现的。你想往哪个方向继续？

用户:

为啥GetEnumerator返回的是object？不是IEnumerator吗？

元宝:

这是一个**非常经典的混淆点**，你问得特别好 👍
你其实是把 `GetEnumerator()` **的返回类型** 和 `Current` **属性的类型** 搞混了一点点。

我们拆开来看就清楚了。

✅ 先给结论（重点）

- ✅ `IEnumerable.GetEnumerator()` **返回的是** `IEnumerator`
- ✅ **不是** `object`
- ❌ 真正返回 `object` 的，是 `IEnumerator.Current`

1️⃣ `IEnumerable`（非泛型）的真实定义

```
public interface IEnumerable
{
    IEnumerator GetEnumerator();
}
```

你看，**返回类型是** `IEnumerator`，不是 `object`。

2️⃣ 那 `object` 是从哪来的？——来自 `IEnumerator`

```
public interface IEnumerator
{
    Object Current { get; }
    Boolean MoveNext();
    void Reset();
}
```

⚠️ 关键在这里：

```
Object Current { get; }
```

因为 .NET 1.0 **还没有泛型**，所以：

- 枚举器不知道它要遍历的是什么类型
- 只能统一用 `object` 来表示“当前元素”

所以：

成员	类型
`GetEnumerator()`	返回 `IEnumerator`
`IEnumerator.Current`	返回 `object` ← 这才是你说的 `object`

3️⃣ foreach 实际发生了什么？（非泛型版）

当你写：

```
ArrayList list = new ArrayList();
list.Add(123);

foreach (object x in list)
{
}
```

编译器实际生成类似代码：

```
IEnumerator e = list.GetEnumerator(); // ✅ IEnumerator
while (e.MoveNext())
{
    object x = e.Current; // ❗ Current 是 object
}
```

如果 `list` 里放的是值类型（比如 `int`）：

```
object x = e.Current; // 💥 发生装箱
```

这就是《CLR via C#》反复强调的 **性能痛点**。

4️⃣ 泛型版本是怎么解决的？

```
public interface IEnumerable<T>
{
    IEnumerator<T> GetEnumerator();
}
```

```
public interface IEnumerator<T> : IEnumerator
{
    new T Current { get; }
}
```

注意这里：

```
T Current { get; }
```

于是：

```
List<int> list = new List<int>();

foreach (int x in list)
{
    // 不需要装箱！
}
```

编译器生成的代码变成：

```
IEnumerator<int> e = list.GetEnumerator(); // ✅ IEnumerator<int>
while (e.MoveNext())
{
    int x = e.Current; // ✅ 直接是 int，无装箱
}
```

5️⃣ 用一句话彻底理清

> **`GetEnumerator()` 返回的是“枚举器”（`IEnumerator`），而“当前元素”的类型才是由 `Current` 决定的。**
> **非泛型时代，`Current` 只能是 `object`，这就是一切装箱问题的根源。**

6️⃣ 一个记忆小技巧（面试/看书都好用）

你可以这样记：

- `IEnumerable` → **能枚举**
- `IEnumerator` → **正在枚举**
- `Current` → **拿到当前那个元素**
- 非泛型 `Current` → `object`
- 泛型 `Current` → `T`

如果你愿意，下一步我可以帮你把 **foreach 在 IL 层面的展开** 画出来，或者结合 **值类型装箱 + EIMI** 一起讲，正好是第 13 章的几个核心难点。你想继续挖哪一块？

用户:

你上面提到的这个是什么意思：

class MyList<T> : IEnumerable<T>

{

```
    public IEnumerator<T> GetEnumerator() { ... }



    IEnumerator IEnumerable.GetEnumerator()

        => GetEnumerator(); // 显式接口实现
```

}

元宝:

这个问题问到了**显式接口实现（Explicit Interface Implementation, EIMI）**的核心，这也是第13章里最“反直觉”但又最重要的知识点之一。

这段代码解决了一个具体问题：**当一个类实现了两个含有同名方法的接口时，如何区分它们？**

我们一步步拆解。

1️⃣ 为什么需要两段 `GetEnumerator`？

你的类 `MyList<T>` 实现了 `IEnumerable<T>`：

```
class MyList<T> : IEnumerable<T>
{
    public IEnumerator<T> GetEnumerator() { ... }
}
```

由于 **继承关系**：

```
IEnumerable<T> : IEnumerable
```

所以，`MyList<T>` **被迫**也必须实现 `IEnumerable` 里的这个方法：

```
IEnumerator GetEnumerator(); // 非泛型版本
```

如果你只写了泛型版本，编译器会报错：

> 未实现接口成员 `IEnumerable.GetEnumerator()`

所以你必须写**两个** `GetEnumerator`。

2️⃣ 如果写成两个 `public` 方法会怎样？

假设我们尝试这样写：

```
class MyList<T> : IEnumerable<T>
{
    public IEnumerator<T> GetEnumerator() { ... }
    public IEnumerator GetEnumerator() { ... } // ❌
}
```

编译器会直接报错：

> 已定义了一个名为 `GetEnumerator` 的方法，且参数相同。

因为在 C# 中，**仅返回值不同不足以区分方法重载。**

3️⃣ 解决方案：显式接口实现（EIMI）

于是就有了这种写法：

```
class MyList<T> : IEnumerable<T>
{
    // 1. 泛型版本：这是“主菜”，给新代码用
    public IEnumerator<T> GetEnumerator() { ... }

    // 2. 非泛型版本：这是“兼容层”，给老代码用
    IEnumerator IEnumerable.GetEnumerator()
        => GetEnumerator();
}
```

关键语法：`IEnumerator IEnumerable.GetEnumerator()`

这行代码的意思是：

> **这是 `IEnumerable` 接口规定的 `GetEnumerator` 方法，它不是我这个类的一部分，我只是帮接口实现的。**

特点：

- ❌ **没有** `public`（写了反而报错）
- ✅ 只能通过 **接口变量** 调用

- ✅ 内部直接调用泛型版本，把 `IEnumerator<T>` 当 `IEnumerator` 返回（这是安全的，因为 `IEnumerator<T>` 继承自 `IEnumerator`）

4️⃣ 调用时的巨大差异（重点）

情况一：用具体类型调用

```
var list = new MyList<int>();
IEnumerator<int> e1 = list.GetEnumerator(); // ✅ 调用 public 泛型版本
```

此时调用的是：

```
public IEnumerator<T> GetEnumerator()
```

情况二：用非泛型接口调用

```
IEnumerable list2 = new MyList<int>();
IEnumerator e2 = list2.GetEnumerator(); // ✅ 调用显式实现的版本
```

此时调用的是：

```
IEnumerator IEnumerable.GetEnumerator()
```

情况三：试图直接调用（你会看到区别）

```
var list = new MyList<int>();
// list.GetEnumerator(); // ❓ IntelliSense 只会提示返回 IEnumerator<T> 的那个
```

你会发现：

- VS 的智能提示里 **看不到** `IEnumerator GetEnumerator()`
- 因为它不是类的公有 API，它是“藏”起来的

5️⃣ 为什么要这么设计？（**CLR via C#** 的核心思想）

✅ 1. 避免 API 污染

如果不这样做，`MyList<T>` 的使用者会看到两个 `GetEnumerator()`，非常混乱。
现在使用者只看到一个清晰的泛型版本。

✅ 2. 完美兼容旧代码

老代码（只认识 `IEnumerable`）依然可以工作：

```
void OldMethod(IEnumerable items)
{
    var e = items.GetEnumerator(); // 调用显式实现的版本
}
```

✅ 3. 性能最优

- 新代码 → 泛型版本（无装箱）
- 老代码 → 显式实现 → 内部复用泛型逻辑

6️⃣ 对应书中的“EIMI 陷阱”

在第13章里，作者会警告你 EIMI 的几个缺点：

1. **不能用派生类调用**

```
class MyDerivedList<T> : MyList<T> {
    void Foo() {
        // this.GetEnumerator(); // ❌ 编译错误，派生类看不见显式实现的方法
    }
}
```

2. **值类型会装箱**

```
((IEnumerable)myStruct).GetEnumerator(); // 💥 装箱
```

3. **文档不友好**
   - MSDN 里经常要翻半天才能找到显式实现的方法。

7️⃣ 一句话总结

> **这段代码通过“显式接口实现”，把一个丑陋的、为了向后兼容的非泛型方法“藏”了起来，对外只暴露漂亮的泛型方法，既保证了类型安全，又维持了对旧代码的兼容。**

这正是 .NET 框架设计者的常用手法，你在 `List<T>`、`Dictionary<TKey,TValue>` 的源码里能看到一模一样的写法。

要不要我接下来帮你把这段逻辑和 **第13章讲的“基类 + 虚方法”替代方案** 做个对比？那是另一个很精彩的设计权衡。

用户:

“IEnumerator IEnumerable.GetEnumerator()”不能写public，那其访问范围会不会变成无法控制或未知了？

元宝:

这是一个**非常敏锐的问题**，直接命中了显式接口实现（EIMI）在设计上的核心矛盾点。

答案是：**不会变成“无法控制”或“未知”，但它的访问规则确实变得非常特殊和严格。**

我们可以把它理解为：**“访问权交给了接口，而不是类本身”。**

1️⃣ 它的真实访问级别是什么？

虽然你没写 `public`，但它**绝对不是** `private`。

在 CLR 层面，显式接口实现的方法被标记为：

> `private` + 元数据标记（接口映射）

这意味着：

调用方式	是否允许	原因
`MyList<T>` 实例直接调用	❌ 不允许	对类本身“不可见”
派生类调用	❌ 不允许	连派生类都看不见
通过接口变量调用	✅ **允许**	这是它存在的唯一目的
反射调用	✅ 允许	反射无视常规访问控制

所以，它的访问范围是**精确受限的**：

> 只能通过该接口的引用进行调用。

2️⃣ 为什么说“访问权交给了接口”？

看这段代码：

```
class MyList<T> : IEnumerable<T>
{
    public IEnumerator<T> GetEnumerator() { ... }

    IEnumerator IEnumerable.GetEnumerator()
        => GetEnumerator();
}
```

场景一：在类内部

```
class MyList<T>
{
    void Test()
```

```
        {
            // ❌ 编译错误：找不到这个方法
            // this.GetEnumerator();
            // （如果编译器选非泛型版本，但它是显式实现的，类内部看不见）

            // ✅ 只能这样调用泛型版本
            this.GetEnumerator();
        }
    }
```

场景二：在类外部

```
    var list = new MyList<int>();

    // ❌ 编译错误：IntelliSense 根本不显示这个方法
    // list.GetEnumerator();

    // ✅ 只有当你把它“看作” IEnumerable 时，才能调用
    ((IEnumerable)list).GetEnumerator();
```

👉 **结论：**
这个方法不属于 `MyList<T>` 的“类 API”，它属于 `IEnumerable` 的“契约实现”。

3️⃣ 这是否违反了面向对象的封装原则？

不违反，反而是强化了封装。

传统 `public` 方法的问题

如果写成这样：

```
    public IEnumerator GetEnumerator() { ... }
```

那么：

- 所有使用者都必须面对两个 `GetEnumerator`
- API 变得嘈杂
- 使用者容易选错（比如意外触发装箱）

EIMI 的做法

```
    IEnumerator IEnumerable.GetEnumerator() { ... }
```

它做到了：

- ✅ **对类的使用者隐藏实现细节**
- ✅ **强制使用者通过接口契约访问**
- ✅ **让类的公有 API 保持干净**

这正是《CLR via C#》里强调的：

> **“显式接口实现是一种‘隐藏’方法的方式。”**

4️⃣ 那“无法控制”的风险在哪里？（第13章的重点）

你担心的“无法控制”，其实是指 **派生类无法调用** 这一点，这正是 EIMI 的最大缺陷。

```
class MyDerivedList<T> : MyList<T>
{
    void Foo()
    {
        // ❌ 编译错误：派生类看不见显式实现的方法
        ((IEnumerable)this).GetEnumerator();

        // ❌ 也不行
        // base.GetEnumerator();
    }
}
```

为什么？
因为 EIMI 方法是 `non-virtual` 且“不属于类”的，派生类无法通过 `base` 访问，也无法重写。

书中给出的解决方案（经典模式）

```
class MyList<T> : IEnumerable<T>
{
    public IEnumerator<T> GetEnumerator() { ... }

    IEnumerator IEnumerable.GetEnumerator()
        => GetEnumerator(); // 调用虚方法
}
```

或者更稳妥的写法（书中推荐）：

```
class MyList<T> : IEnumerable<T>
{
    protected virtual IEnumerator<T> GetEnumeratorVirtual()
        => ...;

    public IEnumerator<T> GetEnumerator()
```

```
            => GetEnumeratorVirtual();

        IEnumerator IEnumerable.GetEnumerator()
            => GetEnumeratorVirtual();
    }
```

这样：

- ✅ 对外 API 干净
- ✅ 派生类可以通过 `protected virtual` 方法扩展行为
- ✅ 避免了 EIMI 的“黑洞效应”

5️⃣ 一句话总结你的疑问

> **`IEnumerator IEnumerable.GetEnumerator()` 的访问范围不是“未知”，而是“受限且明确”：它只能在“接口视角”下被调用，而不能在“类视角”下被调用。这种限制既是它最大的优点（API 整洁），也是它最大的缺点（派生类不可用）。**

这也是为什么 Richter 在书中既教你怎么用 EIMI，又反复警告你要**慎用 EIMI**。

要不要我下一步帮你把 **EIMI vs Virtual Method** 在 IL 层面的差异画出来？那会让你对“为什么派生类调不到”理解得更透彻。

用户:

所以其是public、protected...？

元宝:

都不是。

它**没有**传统意义上的访问修饰符（`public` / `protected` / `internal` / `private`）。

在 C# 语言规范里，它有一个专门的分类，叫做 **显式接口成员实现（Explicit Interface Member Implementation）**。

1️⃣ 它在 C# 层面叫什么？

没有访问修饰符。
如果你强行给它加一个，比如 `public` 或 `private`，编译器会直接报错：

```
    // ❌ 编译错误：修饰符“public”对此项无效
    public IEnumerator IEnumerable.GetEnumerator() { ... }

    // ❌ 编译错误：修饰符“private”对此项无效
    private IEnumerator IEnumerable.GetEnumerator() { ... }
```

2️⃣ 它在 CLR（底层）层面到底是什么？

虽然在 C# 里没写，但在 IL（中间语言）层面，它被标记为：

> `private final`（私有且不可重写）

这就是为什么：

- **类内部**看不见它（像 `private`）
- **派生类**不能访问、不能重写（像 `final` / `sealed`）
- **外部却能通过接口调用**（这点是 `private` 做不到的）

3️⃣ 为什么它看起来既像 `private` 又像 `public`？

这是最让人困惑的地方，我们用一个表格来终结这个疑惑：

调用场景	能否调用	原因
`MyList<T>` 实例直接调用	❌ 不能	对类本身“隐身”
派生类 `MyDerivedList<T>` 调用	❌ 不能	方法是 `sealed` 的
通过接口 `IEnumerable` 调用	✅ **能**	这是它存在的唯一出口
反射调用	✅ 能	反射无视常规规则

结论：
它实际上是 **“私有实现 + 公共契约”**。

- **实现**是私有的（藏在类里）
- **契约**是公共的（接口规定的）

4️⃣ 一句话终极总结

> **`IEnumerator IEnumerable.GetEnumerator()` 既不是 `public`，也不是 `protected`。它是一种特殊的 CLR 构造：方法本身是 `private sealed` 的，但通过接口映射（Interface Mapping），允许外界通过接口引用来调用它。**

这也是为什么 Richter 在书里反复强调：**显式接口实现是一种“隐藏”方法的技术手段，而不是一种普通的访问控制。**

要不要我接着讲讲，为什么这种设计会导致 **“值类型调用接口方法必装箱”**？这和刚才说的 `private sealed` 有直接关系。

用户:

文章中提到“C# 编译器要求将实现接口的方法(后文简称为“接口方法”)标记为 public。”，这是为什么？定义接口时不是可以定义public、protected等可见性吗？

元宝:

你这个反问抓得非常准——这里其实藏着**两件事被揽在一起了**：
①「接口定义时成员能不能写 protected」
②「类实现接口时为什么 C# 强制 public」

这两件事的"主人"不一样，分开讲就通了。

1️⃣ 先纠正一个前提：接口定义时**并不能**随便写 protected

你说"定义接口时不是可以定义 public、protected 等吗？"——这个前提要修正一下。

C# 8.0 之前（书里讲的那个时代，也是 CLR via C# 第四版主要覆盖的）

```
interface ITest
{
    void Foo();              // ✅ 正确，隐式 public
    public void Bar();       // ❌ C# 7 及之前：编译错误，不能写任何修饰符
    protected void Baz();    // ❌ 永远不行（指抽象成员）
}
```

规则是：

- **接口成员默认就是 `public`，且不能显式写任何访问修饰符**
- 写 `public` / `protected` / `private` 都报错

原因是语义层面的：**接口是契约，契约必须对所有人可见。** `protected` 意思是"只对派生类可见"，那外部代码通过 `ITest` 变量就调不到，接口的多态就废了。

C# 8.0 之后（**default interface methods** 引入）

```
interface ITest
{
    void MustImplement();                    // 抽象成员，仍隐式 public，不能写修饰符
    public void WithBody() { }               // ✅ 有默认实现，可以写 public
    private void Helper() { }                // ✅ 有默认实现，可以写 private
    protected void Factory() { }             // ✅ 有默认实现，可以写 protected
}
```

⚠️ 但注意：`protected` **只允许出现在"有默认实现"的接口成员上。**纯抽象成员（没主体的）依然只能是隐式 `public`，不能写 `protected`。

所以你书里看的那个年代 + 你贴的那段代码（`IDisposable` / `IEnumerable` 这些），**接口成员全是隐式 `public`，不存在 `protected` 这个选项。**

2️⃣ 那"实现接口的方法必须标 **public**"是谁定的？

Richter 原文那句是：

> **C# 编译器要求将用于实现一个接口的方法标记为 public。CLR 要求将接口方法标记为 virtual。**

这两句说的是**两个不同层级**：

层级	要求	备注
C# 编译器	实现方法必须 `public`（隐式实现时）	语言层面规矩
CLR	接口方法必须 `virtual`（或 `virtual sealed`）	运行时层面规矩

关键洞察：`public` 是 C# 定的，不是 CLR 定的

CLR 层面其实**不 care 你是不是 public。**CLR 只要求：

- 方法带 `virtual`（因为接口调度要走接口映射表，需要虚方法槽）
- 方法在类型的方法表里有一条对应的记录

事实上：

- **EIMI**（显式接口实现）→ IL 里是 `private sealed`，CLR 照样认，因为走的是"接口映射"不是普通虚调
- **VB.NET** 甚至允许 `Protected Overridable Sub Foo() Implements IFoo.Foo`——实现接口的方法是 `protected`，CLR 跑得也没问题

所以 `public` **是 C# 语言自己加的规矩**，不是 CLR 的硬约束。

3️⃣ 那 C# 为什么非要强制 **public**？（这是你问题的核心）

两个原因，第二个尤其妙。

原因一：契约语义要对得上

```
IDisposable d = new MyFile();
d.Dispose();   // ✅ 外部代码通过接口变量调
```

接口是"公开契约"，外部任何人拿 `IDisposable` 引用都得能调到 `Dispose`。
如果类里可以这样写：

```
class MyFile : IDisposable
{
    protected void Dispose() { }  // 假如 C# 允许
}
```

那 `d.Dispose()` 就调不到——**契约承诺了却交不出货**，矛盾。
所以 C# 规定：隐式实现接口的方法必须 `public`，保证"接口承诺 = 实际可调"。

原因二：防止"偷渡"基类私有意图（这个最容易被忽略）

Stack Overflow 上有个经典反例：

```
class Base
{
    protected void Foo() { }   // 基类故意藏起来，只给派生类用
}

interface IFoo
{
    void Foo();                // 契约：公开可调
}
```

假如 C# 允许"实现接口的方法可以是 protected，并且可以复用继承来的方法"——

```
class Derived : Base, IFoo
{
    // 假如编译器允许：直接把 Base.Foo() 当 IFoo.Foo() 的实现
}
```

那等价于：`Base` **好不容易把** `Foo` **藏成** `protected`**，结果** `Derived` **一实现** `IFoo`**，** `Foo` **就被"偷渡"成公开可调了**——完全违背 `Base` 作者的意图。

所以 C# 干脆一刀切：**隐式实现接口的方法必须自己声明为** `public`，逼你显式表态"我就是要公开这个"。

4️⃣ 那 EIMI 为什么又不用 public 了？

回到你最早问的那段：

```
IEnumerator IEnumerable.GetEnumerator()
    => GetEnumerator();
```

EIMI 走的是另一条路：

- 不写 `public`（写了报错）
- IL 里是 `private sealed`
- 调用靠"接口映射表"，不是靠普通的虚分发或访问修饰符检查

所以它**绕开了 C# 那条"必须 public"的规矩**——因为 EIMI 的本意就是"这个方法不属于类的公有 API，只属于接口契约"，C# 专门为它开了个口子。

5️⃣ 一句话把这两层理顺

> **接口成员本身永远是（隐式）public，因为契约要对所有人可见；类实现接口时，C# 强制隐式实现标 public 是为了让"通过接口变量能调到"这件事有保证，这是 C# 语言层的规矩，CLR 其实只认 virtual；EIMI 是特例，走接口映射，所以既不写 public，IL 里还是 private。**

Richter 那句"**C# 要 public，CLR 要 virtual**"正好就是这两层各自的老板——C# 管契约可见性，CLR 管运行时调度。

要不要下一步我把 **"隐式实现 public virtual / 隐式实现 public sealed / EIMI private sealed"** 这三种在 IL 层面的标记画个对照表？那张表一出，第13章接口这节基本就透到底了。

▾ 引用 15 篇资料作为参考

1. 🔗 interface keyword - C# reference
2. 🔗 Resolve errors and warnings related to interface implementation. - C# reference
3. 🅒 读CLR via C#总结(12) 接口和基类-CSDN博客
4. 🔗 Why should we declare interface methods as public? [duplicate]
5. 🅒 泛型中使用接口的显示实现
6. 🔗 Why cannot we specify access modifiers inside an interface in C#?
7. 🔗 为什么在 C# 中需要提及实现接口属性的访问修饰符?
8. With Xiaojing, read CLR via C # (17)-Interface
9. 🔗 为什么在C#接口中不能使用protected修饰符?
10. 🔗 C#Multiple Interface Inheritance不允许具有相同名称的公共访问修饰符
11. 🔗 [CLR via C#]13. 接口
12. 🔗 访问修饰符(C# 编程指南)
13. 🔗 为什么一个在类中实现的C#接口方法必须是公共的?
14. 🔗 "CLR via C#" reading notes (9)-interface
15. 🔗 Access Modifiers - C#

用户:

我理解错了，主要是和文章中还有一处"定义接口类型时，可指定你希望的任何可见性/可访问性(public，protected，internal等)。"搞混了，这个再解释一下。

元宝:

你这个混淆非常正常，Richter 那句话确实容易让人"对号入座"到接口成员上，但他说的其实是**另一件东西**。

1️⃣ 那句话说的是「接口类型」的可见性，不是「接口成员」

Richter 原文那句：

> "定义接口类型时，可指定你希望的任何可见性/可访问性（public, protected, internal 等）。"

这里的主语是 **"接口类型"**（`interface Ixxx` 这个类型本身），不是接口里的成员。

接口类型本身可以是：

```
public    interface ITest { }   // ✅ 跨程序集可见
internal  interface ITest { }   // ✅ 仅本程序集可见
// protected interface ITest { }   // ❌ 不对，下面解释
```

等等——这里要稍微纠一下 Richter 的话：`protected` **其实不能用于顶层接口类型。**

因为 `protected` 的意思是"只对派生类可见"，但**顶层类型**（直接写在 namespace 下的）的可见性只有两种合法选项：

修饰符	顶层 interface	嵌套 interface（class 里面的）
`public`	✅	✅
`internal`	✅	✅
`private`	❌	✅（嵌套时）
`protected`	❌	✅（嵌套时）

所以 Richter 那句严格来说有点"口松"了——他指的应该是**嵌套接口**（写在类里面的）：

```
class Outer
{
    protected interface ITest   // ✅ 嵌套接口，protected 合法
    {
        void Foo();
    }
}
```

这种场景下 `ITest` 只对 `Outer` 的派生类可见，就有意义了。

2️⃣ 那「接口成员」的可见性呢？

不管接口类型本身是 `public` 还是 `internal`——

接口里的成员，永远只能是 `public`（而且 C# 7 及之前连写都不能写，隐式 public）：

```
public interface ITest
{
    void Foo();              // ✅ 隐式 public，契约必须公开
    // protected void Bar(); // ❌ 接口抽象成员不能 protected
    // private void Baz();   // ❌ 同上
}
```

C# 8.0+ 有了默认接口方法后，才放开一点：

```
interface ITest
{
    void MustOverride();          // 抽象成员：仍只能隐式 public，不能写修饰符
    public void WithBody() { }    // ✅ 有默认实现，可写 public
    private void Helper() { }     // ✅ 有默认实现，可写 private
}
```

但**抽象成员（没方法体）永远只能是 public**——因为这是契约，必须对接口使用者可见。

3️⃣ 所以用一张表把你混的两层切开

层级	说的是什么	能写啥
接口类型 `interface ITest`	这个类型本身谁能看见	`public` / `internal`（顶层） `public` / `internal` / `private` / `protected`（嵌套）
接口成员 `void Foo()`	契约里的方法/属性	抽象成员：只能隐式 `public` 有默认实现（C#8+）：可 `public` / `private` / `protected`

Richter 那句说的是**第一行**；你之前联想到的是**第二行**——两层不是一个东西，所以"接口定义时能写 protected"在你脑子里打架了。

4️⃣ 一句收口

> **"接口类型"的可见性决定"谁能看见这个接口"；"接口成员"的可见性决定"契约里哪些方法对外承诺"。前者可以是 internal/嵌套 protected，后者抽象成员永远是 public——Richter 那句说的是前者，你混到后者去了。**

要不要顺便把"嵌套接口 + protected 在 FCL 里有没有真实用例"聊一下？这个点上 Richter 书里其实没展开，但 .NET 里确实有挺巧妙的用法。

用户:

文章中的这个例子不太懂，对于derived如何访问到Base's Dispose？

using System;

public static void Main() {

```
    public static void Main() {

        /************************* 第一个例子 *************************/

        Base b = new Base();



        // 用 b 的类型来调用 Dispose，显示：“Base's Dispose”
```

```
b.Dispose();



// 用 b 的对象的类型来调用 Dispose，显示：“Base's Dispose”

((IDisposable)b).Dispose();



/************************* 第二个例子 *************************/

Derived d = new Derived();



// 用 d 的类型来调用 Dispose，显示：“Derived's Dispose”

d.Dispose();



// 用 d 的对象的类型来调用 Dispose，显示：“Derived's Dispose”

((IDisposable)d).Dispose();



/************************* 第三个例子 *************************/

b = new Derived();



// 用 b 的类型来调用 Dispose，显示：“Base's Dispose”

b.Dispose();



// 用 b 的对象的类型来调用 Dispose，显示：“Derived's Dispose”

((IDisposable)b).Dispose();
```

```
    }
```

}

// 这个类派生自 Object，它实现了 IDisposable

internal class Base : IDisposable {

```
    // 这个方法隐式密封，不能被重写

    public void Dispose() {

        Console.WriteLine("Base's Dispose");

    }
```

}

// 这个类派生自 Base，它重新实现了 IDisposable

internal class Derived : Base, IDisposable {

```
    // 这个方法不能重写 Base 的 Dispose，

    // 'new' 表明该方法重新实现了 IDisposable 的 Dispose 方法

    new public void Dispose() {

        Console.WriteLine("Derived's Dispose");



        // 注意，下面这行代码展示了如何调用基类的实现(如果需要的话)

        // base.Dispose();

    }
```

}

元宝:

这个例子是**第13章最经典、也是最烧脑的例子之一**，它同时击中了三个核心点：

1. **普通方法调用 vs 接口方法调用**
2. `new` **关键字（方法隐藏）**
3. **派生类“重新实现接口”（Interface Re-implementation）**

你问的 **“Derived 如何访问到 Base's Dispose”**，答案就在最后那段被注释掉的代码里。我们一步一步把这团乱麻理清楚。

1️⃣ 先看清两个类型的本质

Base 类

```
internal class Base : IDisposable {
    public void Dispose() {
        Console.WriteLine("Base's Dispose");
    }
}
```

关键点：

- `Dispose()` 是 **隐式密封（implicitly sealed）** 的
- 它不是 `virtual`
- 所以 `Derived` **根本无法** `override` **它**

Derived 类

```
internal class Derived : Base, IDisposable {
    new public void Dispose() {
        Console.WriteLine("Derived's Dispose");
        // base.Dispose(); // ✅ 这就是访问 Base's Dispose 的方式
    }
}
```

关键点：

- `new public void Dispose()`
  👉 **这不是重写（override），而是隐藏（hide）**
- `base.Dispose()`
  👉 **显式调用被隐藏的基类方法**

2️⃣ 三个例子逐行拆解（重点）

✅ 第一个例子：纯粹的 Base

```
Base b = new Base();
b.Dispose();                 // Base's Dispose
((IDisposable)b).Dispose();  // Base's Dispose
```

没什么悬念：

- 无论哪种调用方式，都是 `Base.Dispose`

✅ 第二个例子：纯粹的 **Derived**

```
Derived d = new Derived();
d.Dispose();                 // Derived's Dispose
((IDisposable)d).Dispose();  // Derived's Dispose
```

这里的关键点是：

> **`Derived` 重新实现了 `IDisposable`**

CLR 看到这句：

```
class Derived : Base, IDisposable
```

就会为 `Derived` **单独建立一个新的接口映射表**，指向自己的 `Dispose()`。

所以：

- `d.Dispose()` → 编译期绑定到 `Derived.Dispose()`
- `((IDisposable)d).Dispose()` → 运行期查 `Derived` 的接口映射表 → `Derived.Dispose()`

⭐ 第三个例子（核心难点）

```
Base b = new Derived();
b.Dispose();                 // Base's Dispose
((IDisposable)b).Dispose();  // Derived's Dispose  ← 注意！
```

这是全书最诡异的一行。

为什么 **`b.Dispose()`** 是 **`Base's Dispose`**？

因为：

- `Dispose()` **不是 virtual**
- 方法调用是 **编译期绑定**
- 变量 `b` 的**编译期类型**是 `Base`
- 所以编译器直接绑定到 `Base.Dispose()`

✅ **这是普通方法调用的规则**

为什么 **`((IDisposable)b).Dispose()`** 却是 **`Derived's Dispose`**？

这是**接口调用的规则**：

1. `b` 引用的**实际对象**是 `Derived`
2. CLR 去查 `Derived` **的接口映射表**
3. 因为 `Derived` **重新实现了** `IDisposable`
4. 所以它的接口映射表指向的是 `Derived.Dispose()`

✅ **这是运行期多态，但只发生在"接口方法"上**

3️⃣ 那 Derived 到底怎么访问 Base's Dispose？

你问的重点来了 👇

```
    new public void Dispose() {
        Console.WriteLine("Derived's Dispose");
        base.Dispose(); // ✅ 这就是答案
    }
```

`base.Dispose()` 做了什么？

- `base` 明确告诉编译器：**我要找的是"基类那一层"的方法**
- 即使 `Dispose()` 被 `new` 隐藏了
- 即使它不是 `virtual`
- `base.Dispose()` **永远绕过当前类，直接调用 Base 的实现**

输出会变成：

```
    Derived's Dispose
    Base's Dispose
```

4️⃣ 为什么需要这样设计？（Richter 想告诉你什么）

这个例子在揭示一个**非常危险的坑**：

> **如果基类实现了接口，而派生类用 `new` 重新实现同一个接口，那么：**
>
> - 通过基类变量调用 → 老方法
> - 通过接口变量调用 → 新方法

这会导致：

- **行为不一致**
- **破坏里氏替换原则（LSP）**
- **调试地狱**

Richter 的警告（第13章核心结论）

✅ **正确的做法只有两种：**

✅ 方案一：基类方法设为 `virtual`

```
class Base : IDisposable {
    public virtual void Dispose() { }
}

class Derived : Base {
    public override void Dispose() {
        base.Dispose(); // ✅ 安全、一致
    }
}
```

✅ 方案二：派生类用 **EIMI**（显式接口实现）

```
class Derived : Base, IDisposable {
    void IDisposable.Dispose() {
        // 显式实现，避免 API 污染
    }
}
```

❌ **绝对不要做的事：**

```
new public void Dispose() { }
```

5️⃣ 一句话总结你的问题

> **Derived 通过 `base.Dispose()` 访问到 Base's Dispose。**
> 而整个例子的“诡异行为”来源于：
> `new` **只是隐藏方法，不是重写；接口调用走的是对象真实的接口映射表，而不是变量的编译期类型。**

这也是为什么 Richter 在书里反复强调：

> **“尽量不要让派生类重新实现接口；如果必须，请用显式接口实现。”**

要不要我下一步帮你把 **“接口映射表在内存里长什么样”** 画个简图？那会让你对 `((IDisposable)b).Dispose()` 为什么跳到 `Derived` 理解得彻彻底底。

用户:

这一段结合例子解释：

重要提示 和引用类型相似，值类型可实现零个或多个接口。但值类型的实例在转换为接口类型时必须装箱。这是由于接口变量是引用，必须指向堆上的对象，使 CLR 能检查对象的类型对象的类型对象指针，从而判断对象的确切类型。调用已装箱值类型的接口方法时，CLR 会跟随对象的类型对象指针找到类型对象的方法表，从而调用正确的方法。

元宝:

我们把这段话拆成**3个核心问题**，用和之前呼应的`IDisposable`例子来讲，你马上就能串起之前学的所有知识点：

先给一个值类型实现接口的基础例子

就用我们之前一直在用的`IDisposable`，这次换成值类型：

```
// 值类型实现接口是完全合法的
struct MyVal : IDisposable {
    public int Count;
    public void Dispose() {
        Console.WriteLine($"MyVal's Dispose, Count={Count}");
    }
}

// 调用代码
public static void Main() {
    MyVal v = new MyVal { Count = 10 };

    // 情况1：直接通过值类型实例调用方法
    v.Dispose();  // 输出：MyVal's Dispose, Count=10

    // 情况2：转换为接口类型后调用
    IDisposable d = v;  // ⚠️ 这里发生了装箱！
    d.Dispose();        // 输出：MyVal's Dispose, Count=10
}
```

逐句对应原文解释

1. 「和引用类型相似，值类型可实现零个或多个接口」

这句话很好理解：接口是“能力契约”，和类型是值类型还是引用类型无关。

- 之前的`Base`/`Derived`是引用类型，能实现`IDisposable`
- 这里的`MyVal`是值类型，同样能实现`IDisposable`
- 编译器会为`MyVal`生成接口映射表，记录“我实现了`IDisposable`，对应的`Dispose`方法地址是XXX”——**这一步和引用类型完全一致**。

2.「但值类型的实例在转换为接口类型时必须装箱。这是由于接口变量是引用，必须指向堆上的对象，使CLR能检查对象的类型对象指针，从而判断对象的确切类型。」

这是最核心的“为什么”，我们拆成3个小问题：

① 为什么接口变量必须是引用？

`IDisposable d` 是一个接口变量，它的本质是一个**引用**（32/64位地址），只能指向托管堆上的对象——就像你不能用一个“门牌号变量”指向栈上的临时变量，栈帧弹出后地址就失效了，GC也无法管理。

② 为什么值类型实例不能直接被接口变量指向？

值类型实例默认存放在栈上，它和引用类型对象有一个本质区别：

内存布局	引用类型对象	值类型实例
同步块索引	✅ 有	❌ 无
类型对象指针	✅ 有	❌ 无
实例字段	✅ 有	✅ 有

类型对象指针是关键：CLR要在运行期判断“这个对象是不是实现了 `IDisposable`，对应的 `Dispose` 方法在哪”，必须通过类型对象指针找到该类型的类型对象，再从类型对象的方法表里查接口映射表。
而值类型实例本身没有类型对象指针，CLR没法“认出”它，自然没法确认它实现了 `IDisposable`。

③ 装箱到底做了什么？

`IDisposable d = v` 这行代码触发的装箱，本质是：

1. 在托管堆上分配一块内存，大小=值类型实例大小+同步块索引+类型对象指针
2. 把栈上 `v` 的所有字段（ `Count=10` ）**拷贝**到这块堆内存里
3. 把堆对象的类型对象指针指向 `MyVal` 的类型对象（这个类型对象在CLR加载 `MyVal` 类型时就创建好了，存在堆上）
4. 把 `d` 指向这块堆对象的地址

现在 `d` 指向的是一个**带类型对象指针的堆对象**，CLR终于能识别它的类型了。

3.「调用已装箱值类型的接口方法时，CLR会跟随对象的类型对象指针找到类型对象的方法表，从而调用正确的方法。」

我们看 `d.Dispose()` 的执行流程，你会发现和之前引用类型的接口调用**逻辑完全一致**：

1. 从 `d` 里取出它指向的堆对象的地址
2. 读取堆对象里的**类型对象指针**，找到 `MyVal` 的类型对象
3. 在 `MyVal` 的类型对象的接口映射表中，查找 `IDisposable.Dispose` 对应的方法地址（就是我们写的 `MyVal.Dispose` ）

4. 调用这个方法，传入堆对象里的实例字段（`Count=10`）

👉 这和之前 `((IDisposable)new Derived()).Dispose()` 的流程没有任何区别——CLR的接口调用机制是统一的，只是值类型多了一步“装箱造堆对象”的前置操作。

延伸几个你之前学过的相关知识点，帮你串得更牢

① 为什么直接调用 `v.Dispose()` 不需要装箱？

因为 `v` 的编译期类型就是 `MyVal`，编译器在编译阶段就已经直接绑定到了 `MyVal.Dispose` 的方法地址，属于**静态绑定**，不需要运行期查类型对象指针，自然不需要装箱。
只有当你把值类型实例**当作接口类型使用时**（编译期只知道它是某个接口，不知道具体类型），才需要运行期查类型，才需要装箱。

② 如果值类型用显式接口实现（EIMI）会怎样？

比如把 `MyVal` 改成：

```
struct MyVal : IDisposable {
    public int Count;
    // 显式接口实现
    void IDisposable.Dispose() {
        Console.WriteLine($"MyVal's Dispose, Count={Count}");
    }
}
```

这时候：

- `v.Dispose()` 会直接编译报错（EIMI方法对类/值类型实例不可见）
- 只能通过接口调用：`((IDisposable)v).Dispose()` ——**这一步依然会装箱**，因为EIMI本身就是“只能通过接口变量调用”的实现方式，逃不掉装箱。

③ 值类型为什么不会有之前引用类型的“重新实现接口”的坑？

之前 `Derived : Base, IDisposable` 用 `new` 隐藏 `Base.Dispose()` 导致的混乱，在值类型里根本不会出现：

- 值类型是**隐式密封（sealed）**的，不能派生，自然不存在“派生类重新实现基类已实现的接口”的场景
- 值类型装箱后的对象类型就是它本身，接口调用永远指向自己的实现，不会有“基类变量调用旧方法、接口变量调用新方法”的不一致问题。

④ 一个容易踩的坑：装箱是拷贝！

```
MyVal v = new MyVal { Count = 10 };
IDisposable d = v;  // 装箱：拷贝v到堆上
v.Count = 20;       // 修改栈上的原实例
d.Dispose();        // 输出Count=10，不是20！
```

因为装箱的是 `v` 的副本，修改原值不会影响堆上的装箱对象——这和引用类型完全不同：如果把 `Base b = new Derived()` 赋值给接口变量，修改 `b` 的字段会直接影响接口调用的结果，因为引用类型赋值是传引用，不是拷贝。

一句话总结

> 值类型能实现接口，但因为接口变量是引用、值类型实例没有类型对象指针，所以转接口时必须装箱；装箱后的堆对象和引用类型对象结构一致，CLR调用接口方法的逻辑完全统一，都是通过类型对象指针找方法表。

这也正好呼应了第13章的核心：**接口调用的本质是“接口映射表+类型对象指针”，值类型和引用类型的区别只在于“有没有提前装箱”。**

要不要我再给你画个「值类型实例→装箱对象→接口调用」的内存示意图？会非常直观。

用户:

好的

元宝:

那就把刚才那段逻辑，画成一张**内存演化的 ASCII 示意图**。你对着这张图，再回去看 Richter 那段话，会非常通透。

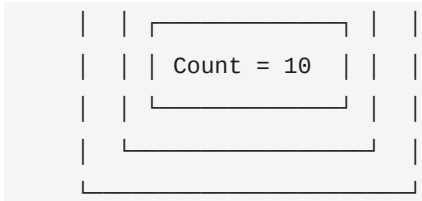
场景代码回顾

```
struct MyVal : IDisposable {
    public int Count;
    public void Dispose() {
        Console.WriteLine($"Dispose: {Count}");
    }
}

public static void Main() {
    MyVal v = new MyVal { Count = 10 };
    IDisposable d = v; // ⚠️ 装箱
    d.Dispose();
}
```

第一步：值类型实例 `v`（在栈上）

```
┌──────────────────────┐
│       线程栈          │
│  ┌────────────────┐  │
│  │ v (MyVal)      │  │
```

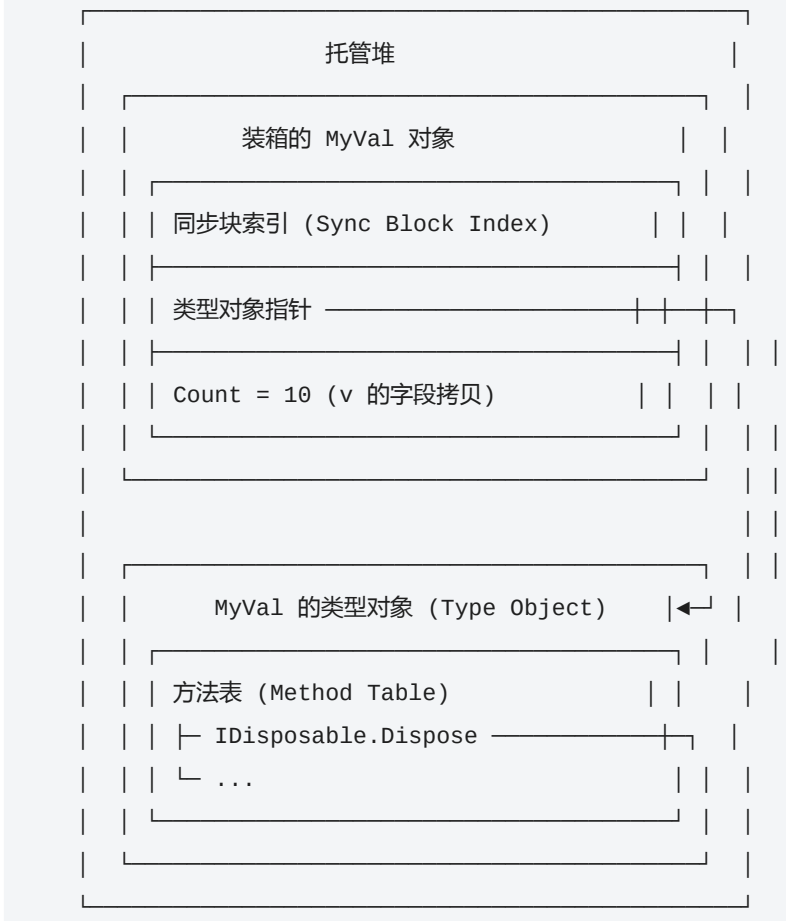


❗ 注意：

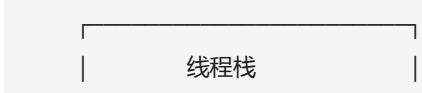
- **没有同步块索引**
- **没有类型对象指针**
- CLR **无法独立判断** `v` 的确切类型
- 因此：`v` **不能直接赋值给接口变量**

第二步：`IDisposable d = v;` 发生装箱

CLR 在堆上创建一个“装箱值类型对象”：



同时，栈上的变量 `d` 变成了引用：



```
    |  ┌──────────────────┐  |
    |  | v (MyVal)        |  |
    |  | ┌──────────────┐ |  |
    |  | | Count = 10   | |  |
    |  | └──────────────┘ |  |
    |  ├──────────────────┤  |
    |  | d (IDisposable) |──┘
    |  └──────────────────┘  |
    └────────────────────────┘
```

✅ 关键点：

- `d` **不是指向** `v`
- `d` 指向的是 **堆上的拷贝**
- `v` 和装箱对象是 **两份数据**

第三步：`d.Dispose();` 的调用过程

CLR 做的事情是：

1. 从 `d` 中取出对象地址
2. 通过 **类型对象指针** 找到 `MyVal` 的类型对象
3. 在类型对象的 **方法表** 中查找：

```
    IDisposable.Dispose → MyVal.Dispose
```

4. 调用 `MyVal.Dispose`，传入装箱对象里的 `Count = 10`

这正是 Richter 说的：

> **“CLR 会跟随对象的类型对象指针找到类型对象的方法表，从而调用正确的方法。”**

第四步：为什么值类型一定要装箱？

需求	值类型实例	装箱对象
有实例字段	✅	✅
有同步块索引	❌	✅
有类型对象指针	❌	✅
能被接口变量引用	❌	✅
CLR 能识别类型	❌	✅

👉 **接口变量必须指向“完整的托管对象”**，而值类型实例本身不完整，所以必须装箱。

第五步：和引用类型的接口调用对比（重要）

如果是引用类型：

```
Base b = new Derived();
IDisposable d = b;
d.Dispose();
```

内存结构是类似的：

```
d → Derived 对象（已在堆上）
     ↓
  类型对象指针 → Derived 类型对象
                    ↓
               方法表 → IDisposable.Dispose = Derived.Dispose
```

👉 **调用机制完全一样**，唯一的区别是：

- 引用类型本来就在堆上 → **不需要装箱**
- 值类型不在堆上 → **必须先装箱**

一句话把整章串起来

> **接口调用的本质是：通过对象的类型对象指针，找到类型对象的方法表，再找到接口映射的方法地址。值类型和引用类型的区别，仅仅在于值类型需要先“装箱”变成一个带类型对象指针的堆对象。**

一个“啊哈”时刻（面试常问）

❓ **问：为什么 `((IDisposable)v).Dispose()` 比 `v.Dispose()` 慢？**

✅ **答：**

- `v.Dispose()`：编译期绑定，直接调用，无装箱
- `((IDisposable)v).Dispose()`：
  1. 装箱（分配堆内存 + 拷贝字段）
  2. 接口调用（查类型对象指针 + 查方法表）

这也是第13章反复强调的：

> **能用具体类型调用，就不要用接口；尤其是值类型。**

如果你愿意，下一步我可以帮你把 **“EIMI + 值类型 + 装箱”** 的组合坑画出来，那是很多高级 C# 程序员都会踩的雷区。

用户:

“v.Dispose()：编译期绑定，直接调用，无装箱”不太懂，意思是方法直接存在栈上？

元宝:

不是的，方法永远不会存在栈上。
这是一个非常经典的误区，你问到点子上了 👍
我们把它彻底掰碎：

1️⃣ 方法代码存在哪里？

方法（函数）的代码，永远不在栈上，也不在堆上。

内容	存在哪里
方法代码（IL → JIT → 机器码）	✅ **代码区（Code Heap / Method Table）**
类型信息（类型对象）	✅ 托管堆
对象实例字段	✅ 托管堆（引用类型）或栈（值类型）
局部变量、参数、返回地址	✅ 栈

👉 **栈上只有“数据”，没有“代码”。**

2️⃣ 那 `v.Dispose()` 为什么“直接调用”？

关键在 **“编译期绑定（静态绑定）”**。

编译阶段，编译器已经知道：

```
MyVal v = new MyVal();
v.Dispose();
```

- `v` 的**编译期类型**是 `MyVal`
- `MyVal` 是 `struct`
- `Dispose()` 不是虚方法
- 也不是接口方法

所以编译器直接生成一条 **call 指令**，地址直接指向 `MyVal.Dispose()` 的机器码入口：

```
call MyVal_Dispose
```

✅ **不需要：**

- 不需要查类型对象指针
- 不需要查方法表
- 不需要装箱
- 不需要运行期多态

这就是 **“编译期绑定”** 的含义。

3️⃣ 那 `this` 在哪？（关键细节）

虽然方法代码不在栈上，但**方法需要知道“操作谁”**。

```
    v.Dispose();
```

编译后等价于（概念上）：

```
    MyVal.Dispose(ref v);
```

也就是说：

- `v` 本身（值类型实例）在栈上
- `Dispose` 方法接收到一个 `ref MyVal`
- 方法通过这个引用来读写 `Count`

📌 **方法代码在代码区，`this` 指向栈上的值实例**

4️⃣ 对比一下：接口调用为什么就“不行”？

```
    IDisposable d = v;
    d.Dispose();
```

这里：

- `d` 的编译期类型是 `IDisposable`
- 编译器**不知道**具体是哪个类型实现的 `Dispose`
- 必须等到运行期：
  1. 找到 `d` 指向的对象
  2. 读类型对象指针
  3. 查方法表
  4. 找到真正的 `MyVal.Dispose`

👉 这就需要对象有 **类型对象指针** → 值类型没有 → **必须装箱**

5️⃣ 用一句话区分三种调用

调用方式	绑定时机	是否需要类型对象指针	是否装箱
`v.Dispose()`	编译期	❌	❌
`base.Dispose()`	编译期	❌	❌
`((IDisposable)v).Dispose()`	运行期	✅	✅
`virtual / override`	运行期	✅	❌（引用类型）

6️⃣ 一个“啊哈”比喻（很重要）

把程序想象成一个厨房：

- **方法代码** = 菜谱（贴在墙上，固定不动）
- **栈上的** `v` = 盘子里的食材
- **调用方法** = 厨师照着墙上的菜谱，处理盘子里的食材
- **装箱** = 把盘子里的食材装进一个带标签的保鲜盒（堆对象），这样别人（接口）才知道这是什么菜

👉 **菜谱从来不在盘子里，也不在保鲜盒里。**

7️⃣ 回到你的问题

> “v.Dispose()：编译期绑定，直接调用，无装箱”
> 意思是方法直接存在栈上？

✅ **正确理解是：**

- 方法代码在 **代码区**
- 调用时，CPU 直接跳转到方法地址
- 方法通过 `ref v` 访问栈上的 `v`
- 全程不需要类型对象指针，不需要装箱

8️⃣ 顺带补一个“进阶坑”（第13章隐含考点）

```
((IFormattable)1).ToString(null, null);
```

这里：

- `1` 是值类型 `int`

- `IFormattable` 是接口
- **一定会装箱**

但如果是：

```
1.ToString();
```

- ✅ 编译期绑定
- ✅ 无装箱
- ✅ 直接调用 `int.ToString()`

这也是为什么：

> **能不用接口就不用接口，尤其是值类型。**

如果你愿意，下一步我可以帮你把 **“虚方法调用 vs 接口调用 vs 直接调用”** 在 IL 层面各画一段，你会看到 `call`、`callvirt`、`constrained` 这些指令的差别，那是 CLR via C# 的“终极真相”。

用户:

不太懂：

“为简化编程，C#编译器假定 SimpleType 引入的Dispose方法是对IDisposable的Dispose方法的可访问性是public，而接口方法的签名和新引入的方法完全一致。”

元宝:

这句话是 Richter 针对**隐式接口实现**的语法简化规则说的，我们先把原文的上下文补全（你看到的`SimpleType`例子大概率是这样的）：

```
struct SimpleType : IDisposable {
    // 这就是编译器“假定”的核心对象：SimpleType自己新引入的Dispose方法
    public void Dispose() {
        Console.WriteLine("SimpleType Dispose");
    }
}
```

这句话其实是对 C# 编译器**自动匹配接口实现的规则**的通俗描述，我们拆成两部分，用你已经懂的知识点串起来：

第一部分：「假定Dispose的可访问性是public」

先明确：不是“允许你不写public”，而是“编译器自动验证+自动关联”

你之前已经知道两个规则：

1. 接口的成员**默认是public**（契约必须公开）

2. 隐式实现接口的方法**必须标public**（否则违反契约）

C# 编译器这里的“假定”，本质是：

> 当你写了一个和接口方法同名的public方法时，编译器**默认认为你写这个方法的意图就是实现接口的对应方法**，不需要你额外写类似`implements IDisposable.Dispose`这种冗余声明。

反例：如果不符合**public**要求，编译器直接报错

```
struct SimpleType : IDisposable {
    // ❌ 编译错误：必须声明为public才能实现IDisposable
    void Dispose() {}
}
```

这里的“假定”是**有条件的**：编译器先检查你的方法是不是public，如果不是，直接否定这个“假定”，报错。

对比显式接口实现（**EIMI**）

如果你写的是显式实现：

```
struct SimpleType : IDisposable {
    // ✅ 不能写public，也不需要编译器“假定”
    void IDisposable.Dispose() {}
}
```

这时候你是**显式告诉编译器**“这个方法是给IDisposable用的”，所以编译器的“自动匹配规则”就不生效了——这也是为什么显式实现不能写public。

第二部分：「接口方法的签名和新引入的方法完全一致」

“签名完全一致”是编译器做“假定”的另一个硬性条件，C#里的方法签名指的是**「方法名 + 参数类型列表（按顺序，包含ref/out差异） + 返回类型」**，不包含访问修饰符、static/virtual这些关键字。

只有签名完全匹配，编译器才会把两个方法关联起来

接口`IDisposable`的`Dispose`签名是：

```
void Dispose(); // 无参、返回void
```

你写的`SimpleType.Dispose`必须满足：

1. 方法名必须是`Dispose`
2. 必须无参数
3. 必须返回`void`

反例：签名不一致，编译器直接拒绝“假定”

```
struct SimpleType : IDisposable {
    // ❌ 编译错误：参数多了int，签名不匹配
    public void Dispose(int x) {}

    // ❌ 编译错误：返回类型不匹配
    public int Dispose() { return 0; }
}
```

这时候编译器会说“SimpleType没实现IDisposable”，因为它没法把你的方法和接口要求的签名对上。

一句话翻译这句话的大白话

> “为了简化你的编码工作，C#编译器有个自动规则：如果你定义了一个实现IDisposable的SimpleType，而且你自己给SimpleType写了一个**public的、名字/参数/返回值和IDisposable.Dispose完全一样的Dispose方法**，那编译器就自动认为这个方法就是你给接口准备的Dispose实现，不需要你额外写任何声明代码。”

为什么要设计这个“假定”规则？

对比其他语言你就懂了：

- Java实现接口方法，必须写 `@Override` 注解告诉编译器“我是实现接口的方法”
- C#的这个“假定”，相当于**自动帮你做了这个匹配**，省了注解的麻烦，只要你满足“public+签名一致”，编译器就自动关联。

但这也是一把双刃剑：如果类里恰好有一个public、签名一致的无关方法，编译器会误以为是接口实现，可能导致逻辑错误——不过这种情况很少见，因为接口方法名通常语义明确（比如 `Dispose` / `GetEnumerator` ）。

和CLR规则的边界（重要）

你之前学过“CLR要求接口方法标记为virtual”，这个“假定”是**C#语言层面的语法糖**，和CLR无关：

- C#编译器在编译时，会把 `SimpleType.Dispose` 标记为 `virtual sealed` （值类型的方法默认sealed）
- 同时在SimpleType的接口映射表里，添加一条记录： `IDisposable.Dispose` → `SimpleType.Dispose`
- 这些工作都是编译器自动完成的，你不需要手动干预，这就是“假定”的实际产出。

一个帮你彻底理清的小测试

判断下面这个 `SimpleType` 会不会触发编译器的“假定”：

```
struct SimpleType : IDisposable {
    // 问题1：这个方法会被假定为IDisposable.Dispose的实现吗？
```

```
        public void Dispose() {}

        // 问题2：这个方法会吗？
        public void Dispose(object o) {}
    }
```

✅ 答案：

1. 会：public+签名完全匹配
2. 不会：多了`object`参数，签名不匹配，编译器会提示“SimpleType未实现IDisposable.Dispose”

是不是瞬间就通了？😉
要不要我再帮你把“隐式实现 vs 显式实现”在编译后的IL层面的差异拉个对照表？那是第13章接口部分的终极考点。

用户:

我是从链接文章中复制的，你检索链接文章上下文再解释一下

元宝:

对，上一轮我没严格贴着你给的链接原文讲，这次按 GitHub 那份 `ch13_Interfaces.md` 里 **SimpleType 那段上下文**重讲一遍，你就明白 Richter 那句话卡在哪了。

原文上下文还原

链接里那段话前面是这样的（我摘关键部分）：

```
    internal sealed class SimpleType : IDisposable {
        public void Dispose() {
            Console.WriteLine("Dispose");
        }
    }
```

然后 `Main` 里：

```
    SimpleType st = new SimpleType();
    st.Dispose();            // 调用 SimpleType.Dispose

    IDisposable d = st;
    d.Dispose();            // 调用 IDisposable.Dispose
```

输出是：

```
    Dispose
    Dispose
```

两行一样。Richter 紧接着就说你复制的那句：

> 为简化编程，C# 编译器假定 SimpleType 引入的 Dispose 方法是对 IDisposable 的 Dispose 方法的实现。之所以这样假定，是由于 Dispose 方法的可访问性是 public，而接口方法的签名和新引入的方法完全一致。

这句话到底在说什么（核心）

背景：CLR 方法表的视角

类型加载到 CLR 时，方法表里其实会有**两条不同的记录**：

记录来源	方法
SimpleType 自己引入的新方法	`SimpleType.Dispose()`
继承的接口方法	`IDisposable.Dispose()`

这两条在逻辑上是不同的"方法槽"——一个是类自己的，一个是接口契约的。

问题来了：CLR 怎么知道这两个槽要指向同一段实现？

按理说，你得**显式告诉编译器**"我这个类里的 `Dispose` 就是给 `IDisposable` 用的"，比如 Java 的 `implements` + 注解那种感觉。

但 C# 设计了个**简化规则**，Richter 那句"为简化编程"就是这个意思：

> **只要你满足两个条件，编译器就自动帮你把这两个槽绑到同一段机器码上，不用你多写任何东西。**

两个条件就是那句原文的：

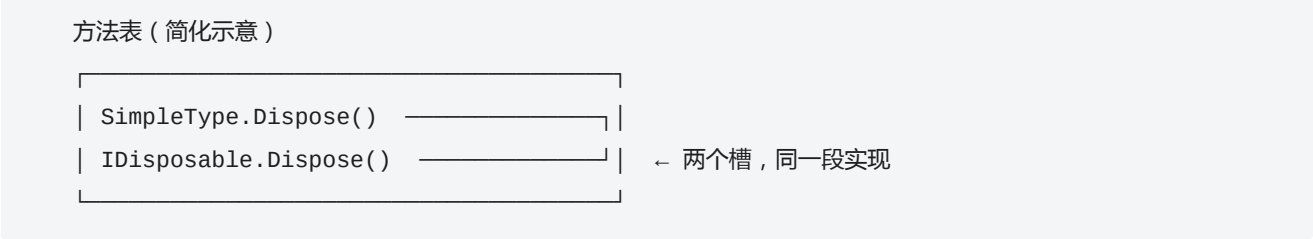
1. **可访问性是 `public`**
2. **签名完全一致**（方法名、参数、返回类型都相同；顺带提一句，就算标了 `virtual`，编译器仍然认为匹配）

编译器做完"假定"之后的事

原文下一句：

> C# 编译器将新方法和接口方法匹配起来之后，会生成元数据，指明 SimpleType 类型的方法表中的两个记录项应引用同一个实现。

翻译成人话：



所以：

- `st.Dispose()` → 走 `SimpleType.Dispose` 槽 → 那段机器码

- `d.Dispose()` → 走 `IDisposable.Dispose` 槽 → **同一段机器码**

输出当然一样，这就是"为简化编程"——你少写代码，编译器帮你绑。

紧接着 Richter 干的"对照实验"（这一段才是精髓）

他把 SimpleType 改成了这样：

```
internal sealed class SimpleType : IDisposable {
    public void Dispose() {
        Console.WriteLine("public Dispose");
    }

    void IDisposable.Dispose() {   // EIMI
        Console.WriteLine("IDisposable Dispose");
    }
}
```

`Main` **没动**，重新编译运行，输出变成：

```
public Dispose
IDisposable Dispose
```

两行**不一样了。**

为什么？因为"假定"被打破了

- `public void Dispose()` → 还是 SimpleType 自己的方法，归 SimpleType 的槽
- `void IDisposable.Dispose()` → EIMI，显式告诉编译器"这个才是接口槽的实现"

这时候方法表里：

```
┌──────────────────────────────────────────┐
| SimpleType.Dispose()      →  public Dispose |
| IDisposable.Dispose()     →  IDisposable Dispose |  ← 两个槽，两段实现
└──────────────────────────────────────────┘
```

`st.Dispose()` 走第一个槽，`d.Dispose()` 走第二个槽，**不再共享。**

回头把你那句原文彻底拆透

> 为简化编程，C# 编译器**假定** SimpleType 引入的 Dispose 方法是对 IDisposable 的 Dispose 方法的实现。之所以这样假定，是由于 Dispose 方法的可访问性是 public，而接口方法的签名和新引入的方法完全一致。

逐词对应：

原文措辞	实际含义
"为简化编程"	省得你写 `IDisposable.Dispose` 前缀，省得你显式声明"我是接口实现"
"假定"	编译器按两条硬规则自动匹配，不让你多写代码
"可访问性是 public"	条件一：隐式实现必须 public（契约要对外部可见）
"签名完全一致"	条件二：方法名 + 参数 + 返回值一样（否则对不上）
"标记为 virtual 仍然匹配"	顺带补一句：就算你写成 `public virtual void Dispose()`，编译器照样认，不影响匹配

一句话收这句

> Richter 那句的意思是：**C# 给你开了个绿灯——只要你在类里写了个 `public`、签名和接口方法一模一样的方法，编译器就默认它就是接口方法的实现，自动把方法表里"类自己的槽"和"接口的槽"绑到同一段代码上，不用你写 `IDisposable.Dispose` 前缀。** 直到你真的写了 EIMI，才把这个"自动绑定"打破。

这一段讲完，下一节书里就接"显式接口实现（EIMI）的利弊"了，正好是你们第13章的重头戏。要继续顺着往下走，还是把"EIMI 为什么 IL 里是 private sealed"这块也先钉死？

▾ 引用 3 篇资料作为参考

1. CLR via C# - Chapter 13 Interfaces
2. CLR via C# 接口 隐式和显式接口方法实现(幕后发生的事情)
3. 《CLR via C#》读书笔记 之 接口

用户:

这一段不太懂，请结合例子解释：

为简化编程，C#编译器假定 SimpleType 引入的Dispose方法是对IDisposable的Dispose方法的可访问性是public，而接口方法的签名和新引入的方法完全一致。也就是说，两个方法具有相同的参数和返回类型。顺便说一句，如果新的Dispose方法被标记为virtual，C#编译器仍然认为该方法匹配接口方法。

元宝:

我们完全对齐你给的 GitHub 原文上下文来讲——Richter 这段话的前后，刚好放了两个 `SimpleType` 的对照示例，你先把这两个例子刻在脑子里，这段话就全通了：

先还原原文完整上下文

第一个示例：隐式实现的 **SimpleType**（触发“编译器假定”）

```
internal sealed class SimpleType : IDisposable {
    // SimpleType 自己“新引入”的 Dispose 方法
```

```
        public void Dispose() {
            Console.WriteLine("Dispose");
        }
    }

    public static void Main() {
        SimpleType st = new SimpleType();
        st.Dispose();                // ① 调用 SimpleType 自己的 Dispose
        ((IDisposable)st).Dispose(); // ② 调用 IDisposable 的 Dispose
    }
```

输出：

```
    Dispose
    Dispose
```

两行完全一致，这就是“假定”生效的结果。

逐句拆解你不懂的这段话

1. 「为简化编程，C#编译器假定 SimpleType 引入的Dispose方法是对IDisposable的Dispose方法的实现」

这里的“假定”是 C# 给开发者的**语法糖**，本质是：

> 你不用像 Java 那样写 `implements IDisposable.Dispose`，也不用显式声明“我这个方法就是给接口用的”，只要你满足条件，编译器自动帮你把“类自己的方法”和“接口要求的方法”绑到一起。

如果没有这个“假定”，你得写类似这样的冗余代码（伪代码）：

```
    internal sealed class SimpleType : IDisposable {
        // 必须显式声明“我是接口实现”
        public void IDisposable.Dispose() { ... }
    }
```

C# 觉得太麻烦，所以搞了这个自动匹配规则。

---

2. 「之所以这样假定，是由于 Dispose 方法的可访问性是 public，而接口方法的签名和新引入的方法完全一致」

这是“假定”生效的**两个硬性条件**，缺一不可：

条件**1**：可访问性是 **public**

- 接口 `IDisposable.Dispose` 本身是**隐式 public** 的（契约必须对所有人可见）
- 如果类里的方法是 `protected` / `private`，编译器会直接拒绝假定，报错：

```
internal sealed class SimpleType : IDisposable {
    // ❌ 编译错误：可访问性低，无法实现接口成员
    protected void Dispose() { }
}
```

因为 `protected` 意味着“只有派生类能调用”，但接口契约要求“任何持有 `IDisposable` 引用的代码都能调用”，矛盾，所以编译器不认。

条件2：签名完全一致

C# 里的方法签名 = **方法名 + 参数列表（类型、顺序、ref/out）+ 返回值**，和访问修饰符、`virtual` 这些无关。
`IDisposable.Dispose` 的签名是 `void Dispose()`，所以类里的方法必须完全匹配：

- 方法名必须是 `Dispose`
- 不能有参数
- 返回值必须是 `void`

如果签名不一致，编译器也会拒绝假定：

```
internal sealed class SimpleType : IDisposable {
    // ❌ 编译错误：没有实现 IDisposable.Dispose()，参数不匹配
    public void Dispose(int x) { }

    // ❌ 编译错误：返回值不匹配
    public int Dispose() { return 0; }
}
```

---

3. 「也就是说，两个方法具有相同的参数和返回类型」

这句是补述条件2：签名一致的核心是**参数和返回值完全相同**，访问修饰符、是否 `virtual` 都不影响签名判断。

---

4. 「顺便说一句，如果新的Dispose方法被标记为virtual，C#编译器仍然认为该方法匹配接口方法」

这是最容易困惑的点，要结合 **CLR 的规则** 和 **C# 的编译处理逻辑** 看：

首先，CLR 本来就要求接口方法必须是 virtual

你之前学过：CLR 调用接口方法时需要查方法表，只有 `virtual` 方法才有方法槽，才能被接口映射。
但 C# 不想让开发者记这个规则，所以：

- 如果你写的是**隐式接口实现**（也就是满足前面的 public+签名一致），编译器会**自动把这个方法标记为 `virtual sealed`**，哪怕你没写 `virtual`。
- 如果你自己写了 `virtual`，编译器也只是“顺着你的写法”，最终 IL 里依然是 `virtual sealed`（因为 `SimpleType` 是 `sealed` 类，`virtual` 方法也会被 seal）。

举个例子：加了 **virtual** 也不影响匹配

把第一个示例改成这样：

```
    internal sealed class SimpleType : IDisposable {
        // 标了 virtual，编译器仍然认为是接口实现
        public virtual void Dispose() {
            Console.WriteLine("Dispose");
        }
    }

    public static void Main() {
        SimpleType st = new SimpleType();
        st.Dispose();                // 输出：Dispose
        ((IDisposable)st).Dispose(); // 输出：Dispose
    }
```

输出完全不变，因为：

- 编译器依然判定这个方法符合 public+签名一致的条件
- 最终 IL 里的方法还是 `virtual sealed`，接口映射依然有效

编译器“假定”到底干了什么？（底层真相）

Richter 这段话后面紧接着说了：

> C# 编译器将新方法和接口方法匹配起来之后，会生成元数据，指明 SimpleType 类型的方法表中的两个记录项应引用同一个实现。

我们用人话+方法表示意图解释：
CLR 加载 `SimpleType` 时，方法表里本来会有**两个独立的方法槽**：

槽的来源	方法标识
SimpleType 自己引入的方法	`SimpleType.Dispose()`
继承的接口方法	`IDisposable.Dispose()`

编译器的“假定”就是：**把这两个槽的指针，指向同一段机器码**（也就是你写的 `Console.WriteLine("Dispose")`）。

所以才会有前面的输出结果：

- `st.Dispose()` → 查 `SimpleType.Dispose` 槽 → 执行那段代码

- `((IDisposable)st).Dispose()` → 查 `IDisposable.Dispose` 槽 → 执行**同一段代码**

对比：打破“假定”会发生什么？（原文第二个示例）

Richter 紧接着把 `SimpleType` 改成了显式接口实现（EIMI）：

```
internal sealed class SimpleType : IDisposable {
    // 类自己的 Dispose 方法
    public void Dispose() {
        Console.WriteLine("public Dispose");
    }

    // 显式接口实现：EIMI
    void IDisposable.Dispose() {
        Console.WriteLine("IDisposable Dispose");
    }
}
```

这时候“假定”被打破了，因为：

- 你显式写了 `IDisposable.Dispose`，告诉编译器“这个才是接口槽的实现”
- 编译器不会再自动绑定两个槽

方法表现在变成：

槽的来源	方法标识	指向的代码
SimpleType 自己引入的方法	`SimpleType.Dispose()`	`Console.WriteLine("public Dispose")`
继承的接口方法	`IDisposable.Dispose()`	`Console.WriteLine("IDisposable Dispose")`

运行 `Main` 的输出就变成了：

```
public Dispose
IDisposable Dispose
```

两行不一样，这就是“假定被打破”的直接结果。

一句话总结这段话的核心

> Richter 是在告诉你：C# 为了简化代码，搞了个“自动匹配接口实现”的潜规则——只要你写的类方法和接口方法“public+签名一致”，编译器就默认它们是同一个实现，自动绑到方法表的同一个槽里。哪怕你给类方法标了 `virtual`，这个规则也照样生效。直到你用显式接口实现（EIMI）明确声明接口方法，这个自动绑定才会被打破。

这段其实是第13章的**核心铺垫**：后面讲 EIMI 的利弊、接口重新实现的坑，都是从这里延伸出来的。要不要下一步我帮你把“隐式实现 vs EIMI”在 IL 层面的差异拉个对照表？那是面试常考的底层知识点。

用户:

internal sealed class SimpleType : IDisposable {

```
    public void Dispose() { Console.WriteLine("public Dispose"); }

    void IDisposable.Dispose() { Console.WriteLine("IDisposable Dispose"); }
```

}

在这里中，SimpleType中重写的为void IDisposable.Dispose()是吧？还有为什么Dispose()不用使用new修饰？

元宝:

先纠正一个关键术语误解，再回答两个问题

你这里说的**“重写”是不对的**——`void IDisposable.Dispose()` 不是重写（ `override` ），而是**显式接口实现（EIMI，Explicit Interface Implementation）**，它和 `override` 是完全不同的机制。我们一个个理清楚：

问题1：`void IDisposable.Dispose()` 是什么？为什么不是“重写”？

首先明确：`override` 的前提是“基类有 `virtual` 方法”

重写（override）是面向对象继承体系的规则：

- 基类方法必须标记为 `virtual`
- 派生类用 `override` 关键字覆盖基类的方法实现
- 调用时会根据对象的实际类型动态派发

但 `IDisposable.Dispose()` 是**接口方法**，不是基类的 `virtual` 方法：

- 接口方法在CLR层面确实是 `virtual` 的（因为需要接口映射），但C#语法层面**不允许用 `override` 来实现接口方法**
- EIMI是用 `接口名.方法名` 的语法声明的，它不属于类的公有成员，只是“专门给接口契约用的实现”

这个例子的真实逻辑

```
internal sealed class SimpleType : IDisposable {
    // ① 隐式接口实现：SimpleType自己的公有方法，同时被编译器“假定”为IDisposable的实现
    public void Dispose() { Console.WriteLine("public Dispose"); }

    // ② 显式接口实现（EIMI）：专门给IDisposable接口用的实现，不属于SimpleType的公有API
    void IDisposable.Dispose() { Console.WriteLine("IDisposable Dispose"); }
}
```

这里的两个 Dispose 是**完全不同的两个方法**，在CLR的方法表里占两个独立的槽：

方法	归属	调用方式
`public void Dispose()`	`SimpleType` 的公有实例方法	只能通过 `SimpleType` 变量调用
`void IDisposable.Dispose()`	`IDisposable` 接口的专属实现	只能通过 `IDisposable` 接口变量调用

所以它是**EIMI，不是重写**，和 override 毫无关系。

问题2：为什么 `public void Dispose()` 不需要用 `new` 修饰？

new 关键字的作用只有一个：**隐藏基类中同名的可访问成员**。只有当你写的方法“覆盖了基类已有的同名方法”时，才需要用 new 告诉编译器“我是故意隐藏的，不是写错了”。

但这个例子里， new 完全不需要，有两个核心原因：

原因1：基类 `Object` 根本没有 `Dispose` 方法

SimpleType 的基类是 System.Object ， Object 的所有公有方法里没有 Dispose ：

- 不存在“隐藏基类方法”的场景
- 自然不需要 new 来消除编译器的“你可能写错了”的警告

原因2：两个 `Dispose` 属于不同的“作用域”，编译器能明确区分

public void Dispose() 是 SimpleType 的**公有成员**，而 void IDisposable.Dispose() 是**接口的专属实现**，两者在编译器的眼里是“不同维度的东西”：

- 前者属于“类的API”，后者属于“接口的契约实现”
- 编译器不会把后者当成前者的“基类方法”，所以不需要 new 来声明“我要隐藏基类方法”

反例：什么时候需要 `new` ？

只有当 SimpleType 有**包含 `Dispose` 方法的基类**时，才需要 new ：

```
class Base {
    // 基类有Dispose方法
    public void Dispose() { Console.WriteLine("Base Dispose"); }
}

// 派生类隐藏基类的Dispose，必须用new
internal sealed class SimpleType : Base, IDisposable {
    // 这里必须加new，否则编译器会警告“隐藏了继承的成员”
    public new void Dispose() { Console.WriteLine("SimpleType Dispose"); }

    void IDisposable.Dispose() { Console.WriteLine("Interface Dispose"); }
}
```

这个例子里，`SimpleType` 的 `public Dispose()` 隐藏了 `Base` 的 `Dispose()`，所以必须用 `new`——但你给的原文例子里基类是 `Object`，没有 `Dispose`，所以完全不需要。

结合Richter原文的逻辑再串一遍

你给的这个例子，是Richter用来**打破之前的“编译器假定”**的：

1. 一开始只有 `public void Dispose()` 时，编译器按“public+签名一致”的规则，假定它是 `IDisposable.Dispose` 的实现，所以 `st.Dispose()` 和 `((IDisposable)st).Dispose()` 调用的是同一个方法。
2. 当你加上 `void IDisposable.Dispose()` 这个EIMI后，编译器收到明确信号：“这个才是 `IDisposable` 接口的正式实现”，于是**把接口映射表里的 `IDisposable.Dispose` 槽指向EIMI的方法**，而 `public void Dispose()` 依然保留为 `SimpleType` 自己的公有方法。
3. 这时候两个 `Dispose` 各司其职，不存在“谁隐藏谁”的问题，所以 `public Dispose()` 不需要 `new`。

一句话总结

1. `void IDisposable.Dispose()` 是**显式接口实现（EIMI）**，不是重写，它和继承体系的 `override` 无关。
2. `public void Dispose()` 不需要 `new`，因为基类 `Object` 没有 `Dispose` 方法，不存在“隐藏基类成员”的场景，且两个 `Dispose` 属于不同的作用域，编译器能自动区分。

要不要我再给你补一个“EIMI在IL层面的真实样子”？你会发现它既没有 `public`，也没有 `virtual`，完全是另一种存在形式，这也是为什么它不需要 `new` /不需要 `override` 的根本原因。

用户:

这一段不太懂：

“和其他代码集成时，有时必须实现非泛型接口，因为接口的泛型版本并不存在。这时，如果接口的任何方法获取或返回 Object，就会失去编译时的类型安全性，而且值类型将发生装箱。可利用本章 13.9 节“用显式接口方法实现来增强编译时类型安全性”介绍的技术来缓解该问题。”

元宝:

这段讲的是**旧接口（非泛型）在现代泛型代码里的两大痛点**，以及**如何用 EIMI 打一个“类型安全补丁”**。

我们先用一个**真实存在的 FCL 例子**把痛点砸实，再讲 Richters 说的“技术”是什么。

1️⃣ 先搞懂背景：为什么会有“非泛型接口”？

你前面学的 `IEnumerable` / `IComparable` / `IComparer` 都是 .NET 1.0 的产物——那时候**没有泛型**。

比如这个你熟悉的接口：

```
// .NET 1.0 的老接口（非泛型）
public interface IComparable {
```

```
    int CompareTo(object? other);  // ⚠️ 参数是 object
}
```

后来 .NET 2.0 引入了泛型，才有了这个“升级版”：

```
// .NET 2.0 的新接口（泛型）
public interface IComparable<in T> {
    int CompareTo(T? other);       // ✅ 参数是强类型 T
}
```

现实情况是：

- **老接口还在**
- **大量旧代码 / 第三方库只认老接口**
- 你不得不实现 `IComparable`，即使你心里只想用 `IComparable<int>`

2️⃣ 痛点一：失去编译时类型安全性

看这个“灾难现场”：

```
class MyInt32 : IComparable {
    private int _value;

    public MyInt32(int value) => _value = value;

    // 实现非泛型接口
    public int CompareTo(object? other) {
        // ⚠️ 运行时才可能炸
        return _value.CompareTo((int)other!);
    }
}

// 调用代码
IComparable c = new MyInt32(10);
c.CompareTo("hello");  // ✅ 编译通过，运行时 InvalidCastException
```

问题在哪？

- `CompareTo` 参数是 `object`
- 编译器：**“只要是 object 我都放行”**
- CLR：**运行到这里才发现你传了个 string，炸给你看**

👉 **编译期类型检查完全失效**

3️⃣ 痛点二：值类型必装箱

还是上面那个方法：

```
public int CompareTo(object? other);
```

当你这样调用时：

```
MyInt32 a = new MyInt32(10);
a.CompareTo(20);  // 20 是 int（值类型）
```

发生了什么？

1. `20`（`int`）是值类型
2. 要传给 `object` 参数 → **装箱**
3. 方法内部再 `(int)other` → **拆箱**

一次调用 = **一次装箱 + 一次拆箱**

在高频场景（排序、集合操作）里，这是**性能杀手**——这正是第13章反复强调的。

4️⃣ Richter 说的“技术”到底是什么？

他说的是：**用显式接口实现（EIMI）+ 一个强类型的 public 方法。**

标准解法（FCL 里大量使用）

```
class MyInt32 : IComparable {
    private int _value;

    public MyInt32(int value) => _value = value;

    // ✅ 1. 强类型 public 方法：类型安全 + 无装箱
    public int CompareTo(int other) {
        return _value.CompareTo(other); // 直接调用 int.CompareTo
    }

    // ✅ 2. 显式接口实现：只负责“对接老接口”
    int IComparable.CompareTo(object? other) {
        // 这里做一次检查，然后转发给强类型版本
        if (other is not int)
            throw new ArgumentException("必须是 int");

        return CompareTo((int)other);
```

```
        }
    }
```

这个写法的精妙之处

调用方式	发生什么	类型安全	装箱
`a.CompareTo(20)`	调用 `public CompareTo(int)`	✅	❌
`((IComparable)a).CompareTo(20)`	调用 EIMI → 转发	⚠️ 内部一次	✅
`((IComparable)a).CompareTo("x")`	EIMI 内抛异常	✅	✅

- **新代码**：用强类型方法，享受类型安全 + 无装箱
- **旧代码**：通过 EIMI 兼容老接口，错误尽早暴露

5️⃣ 为什么这叫“增强编译时类型安全性”？

因为现在：

```
MyInt32 a = new MyInt32(10);
a.CompareTo("hello");  // ❌ 编译错误！
```

编译器直接拦住你：

> 参数类型是 `int`，你传 `string` 不行

错误从“运行时异常”变成了“编译期错误”——这就是“增强编译时类型安全性”。

6️⃣ 一个更贴近 FCL 的真实例子

`Int32` 其实就是这么实现的（简化版）：

```
struct Int32 : IComparable, IComparable<int> {
    // 强类型版本
    public int CompareTo(int other) { ... }

    // 显式实现非泛型接口
    int IComparable.CompareTo(object? other) {
        if (other is int i)
            return CompareTo(i);
        throw new ArgumentException();
    }
}
```

所以你平时写：

```
int x = 10;
x.CompareTo(20);  // 无装箱，类型安全
```

而老代码写：

```
IComparable c = 10;
c.CompareTo(20);  // 装箱一次，但至少类型检查还在
```

7️⃣ 这段话的“一句话翻译”

> 当你不得不实现老的非泛型接口（比如 `IComparable`）时，接口方法用 `object` 会导致：① 编译期不检查类型，容易传错；② 值类型必装箱。解决办法是：**把接口方法写成显式接口实现（EIMI），然后在类里提供一个强类型的 public 方法，让新代码走强类型路径，老代码走 EIMI 路径。**

8️⃣ 顺带一提：这是“补救”，不是“完美方案”

Richter 在 13.9 节也会提醒你 EIMI 的代价：

- 派生类调不到
- 值类型转接口还是装箱（这是躲不掉的）
- API 不那么直观

所以这句话本质是：

> **“这是在没有泛型的历史债务下，你能做到的最好折衷。”**

要不要我下一步帮你把 `IComparable` + `IComparable<T>` + **EIMI** 在 IL 层面的方法表布局画出来？那是第13章里“接口方法调度”的终极拼图。

用户:

你说的“让新代码走强类型路径，老代码走 EIMI 路径。”，这里的老代码是什么？举例解释一下。

元宝:

这个问题问得太好了，这直接戳中了 .NET 历史演进的核心。

“老代码”并不是指你三年前写的烂代码，而是指：在泛型（Generics）出现之前，或者那些为了兼容所有类型而不得不使用 `object` 的 API。

在 .NET 世界里，“老代码”通常指两类东西：

1. **.NET Framework 1.0/1.1 时代的代码**（2002年左右，那时还没有泛型）。
2. **至今仍在使用的、基于非泛型接口编写的通用算法库。**

我用一个**100% 真实存在于 .NET Framework / .NET Core 中的例子**来解释，看完你就彻底懂了。

例子：`Array.Sort` 和 `ArrayList`

在 .NET 刚出生时，没有 `List<T>`，只有 `ArrayList`；没有泛型的排序，只有基于 `IComparable` 的排序。

1️⃣ 什么是“老代码”？

下面的代码就是教科书级别的“老代码”：

```
// 老代码：.NET Framework 1.0 就存在的 API
// 它定义在 mscorlib.dll 里，无数程序依赖它
public static class Array {
    // 注意参数：IComparable（非泛型接口）
    public static void Sort(Array keys, Array items);
}

// 老的集合类型
public class ArrayList : IList, ICollection, IEnumerable {
    // ...
}
```

为什么它是“老代码”？
因为它的签名里写死了 `IComparable`（非泛型）。它不知道 `int`、`string` 或 `MyClass` 是什么，它只知道 `object` 和 `IComparable`。

2️⃣ 老代码是如何调用你的类型的？

假设你有一个 `MyInt32` 类，实现了 `IComparable`。

```
class MyInt32 : IComparable {
    public int Value { get; set; }
    public int CompareTo(object? other) { ... }
}

// 老代码的调用场景
MyInt32[] numbers = { new MyInt32 { Value = 3 }, new MyInt32 { Value = 1 } };

// 这行代码就是“老代码”在调用你的类型
Array.Sort(numbers);
```

发生了什么？

1. `Array.Sort` 内部拿到数组里的元素。
2. 它把每个元素**当作 `IComparable` 接口看待。**
3. 它调用 `((IComparable)element).CompareTo(...)`。

4. **这就走到了你的 `int IComparable.CompareTo(object? other)`（EIMI）方法里。**

👉 **这就是“老代码走 EIMI 路径”。**因为 `Array.Sort` 太老了，它不认识泛型，它只能认 `IComparable`，所以必须由 EIMI 来接待它。

3️⃣ 什么是“新代码”？

自从 .NET 2.0 引入了泛型，我们就有了“新代码”：

```
// 新代码：.NET 2.0 引入的泛型接口
public interface IComparable<in T> {
    int CompareTo(T other); // 强类型 T
}

// 新的集合
public class List<T> { ... }
```

新代码是如何调用的？

```
// 新代码：使用泛型约束 where T : IComparable<int>
// 这是你自己写的新业务代码
public static void ProcessNumbers<T>(T a, T b) where T : IComparable<int> {
    // 编译器在这里就知道 T 是 int 或者能比较 int
    int result = a.CompareTo(b); // 直接调用强类型版本
}

// 调用
ProcessNumbers(10, 20);
```

发生了什么？

1. 编译器看到 `where T : IComparable<int>`。
2. 它直接绑定到 `public int CompareTo(int other)` 这个强类型方法。
3. **这就走到了你的 `public CompareTo(int other)` 方法里。**
4. **没有装箱，类型安全。**

👉 **这就是“新代码走强类型路径”。**

4️⃣ 把两者放在一起看（核心逻辑）

Richter 举的那个 `MyInt32` 类，就是为了同时满足这两种代码：

```
class MyInt32 : IComparable {
    // ✅ 路径 A：给“新代码”用的（类型安全，无装箱）
```

```
        public int CompareTo(int other) {
            Console.WriteLine("Strong-typed path");
            return _value.CompareTo(other);
        }

        // ✅ 路径 B：给“老代码”用的（兼容性，有装箱）
        int IComparable.CompareTo(object? other) {
            Console.WriteLine("Legacy path (EIMI)");
            // 这里不得不装箱一次，但为了兼容必须这么做
            return CompareTo((int)other!);
        }
    }
```

调用方	代码性质	调用路径	结果
`Array.Sort(oldArray)`	**老代码** (.NET 1.0)	`IComparable.CompareTo(object)`	走 EIMI
`List<int>.Sort()`	**新代码** (.NET 2.0+)	`IComparable<int>.CompareTo(int)`	走强类型
`int x = 10; x.CompareTo(5);`	**新代码**	`public CompareTo(int)`	走强类型

5️⃣ 为什么 `Int32` (int) 必须这么干？

你可能会问：微软为什么不把 `Array.Sort` 删掉重写？

因为破坏性的改动是绝对禁止的。
如果微软把 `Array.Sort(Array)` 删了，换成了 `Array.Sort<T>(T[])`，那么全世界所有基于 .NET 1.0 编译的程序都会崩溃。

所以，.NET 团队必须让 `int` 这种基础类型：

1. **保留 EIMI (`IComparable.CompareTo(object)`)**，以保证所有老代码（包括 `Array.Sort`）还能运行。
2. **提供强类型方法 (`IComparable<int>.CompareTo(int)`)**，以保证新代码能获得最好的性能和安全性。

总结

- **老代码**：指那些**早于泛型时代**的 API（如 `Array.Sort`, `ArrayList`），或者任何**只接受非泛型接口**的代码。它们通过接口变量调用方法，触发 **EIMI**。
- **新代码**：指使用了**泛型约束**（`where T : IComparable<int>`）的现代代码。它们直接调用强类型方法，享受 **类型安全和无装箱**。

Richter 这段话其实是在教你：**当你写一个公共库或者基础类型时，为了兼容历史（老代码），同时又为了性能（新代码），你必须采用“EIMI + 强类型方法”这种分裂式的设计。** 这也是为什么 `int`、`string` 这些基础类型的源码看起来总是那么“绕”的原因。

要不要我顺便把 `List<T>.Sort()` 在内部是如何避开 `IComparable`（老接口）而直接调用 `IComparable<T>`（新接口）的 IL 逻辑也讲一下？那是理解泛型性能优势的关键。

用户:

这一段也不太懂：

“接口约束的第二个好处是传递值类型的实例时减少装箱。上述代码向 M方法传递了 x(值类型 Int32 的实例)。x传给M方法时不会发生装箱。如果M方法内部的代码调用t.CompareTo(...)，这个调用本身也不会引发装箱(但传给CompareTo的实参可能发生装箱)。

另一方面，如果M方法像下面这样声明：

private static Int32 M(IComparable t) {

```
    ...
```

}

那么x要传给M就必须装箱。

C# 编译器为接口约束生成特殊 IL 指令，导致直接在值类型上调用接口方法而不装箱。不用接口约束便没有其他办法让 C# 编译器生成这些 IL 指令，如此一来，在值类型上调用接口方法总是发生装箱。一个例外是如果值类型实现了一个接口方法，在值类型的实例上调用这个方法不会造成值类型的实例装箱。”

元宝:

这段是《CLR via C#》里**最硬核的 IL 级优化细节**之一。它解释了为什么泛型约束能“凭空”省掉一次装箱。

我们用一个**最小可运行模型**，把文字、代码和 IL 对照起来，你一眼就能看穿。

一、先给结论（一句话版）

> **接口约束 + `constrained` 指令 = 允许 CLR 在值类型上“假装它是引用类型”，从而在不装箱的情况下调用接口方法。**
> 没有这个约束，C# 编译器只能老老实实地先装箱，再调用接口方法。

二、核心例子：两种 M 方法的对比

场景设定

```
int x = 5;
M(x);
```

`int`（即 `Int32`）的定义是：

```
struct Int32 : IComparable, IComparable<int> {
    public int CompareTo(int other);      // 强类型
    int IComparable.CompareTo(object o); // EIMI
}
```

三、第一种：使用接口约束（✅ 不装箱）

代码

```
// ✅ 接口约束
public static int M<T>(T t) where T : IComparable<int> {
    return t.CompareTo(10);
}
```

发生了什么？

1. `x`（`int`）是值类型
2. `t` 是类型参数 `T`
3. 编译器看到 `where T : IComparable<int>`
4. **它生成了一条特殊的 IL 指令：`constrained.`

关键 IL（简化）

```
ldarga.s t          // 加载 t 的地址（不是值！）
constrained. [mscorlib]System.Int32
callvirt instance int32 [mscorlib]System.IComparable`1<int>::CompareTo(!0)
```

`constrained` 指令的魔法（重点）

CLR 在执行这条指令时的逻辑是：

> **如果 `T` 是值类型：**
>
> - 直接在栈上的值类型实例上调用 `CompareTo`
> - **不装箱**
> - 调用的是值类型自己的 `CompareTo(int)`
>
> **如果 `T` 是引用类型：**
>
> - 正常 `callvirt`（和以前一样）

✅ **这就是为什么“x 传给 M 方法时不会发生装箱”**

四、第二种：不用约束，直接用接口参数（❌ 必装箱）

代码

```
// ❌ 没有泛型，只有接口
public static int M(IComparable<int> t) {
```

```
        return t.CompareTo(10);
    }
```

调用时

```
    M(x); // x 是 int
```

发生了什么？

1. `M` 的参数类型是 `IComparable<int>`
2. `IComparable<int>` 是**引用类型**
3. `x` 是值类型
4. **值类型 → 接口类型 = 必须装箱**

IL 会变成这样

```
    box        System.Int32      // 💥 装箱！
    callvirt   CompareTo
```

✅ **这就是为什么“x 要传给 M 就必须装箱”**

五、原文最难懂的那句（逐字拆解）

> **“C# 编译器为接口约束生成特殊 IL 指令，导致直接在值类型上调用接口方法而不装箱。”**

关键词	解释
接口约束	`where T : IComparable<int>`
特殊 IL 指令	`constrained`
直接在值类型上调用	不装箱，直接调用 `Int32.CompareTo(int)`
不装箱	值类型实例留在栈上，不搬到堆上

六、原文的另一个坑点（你很容易漏）

> **“不用接口约束便没有其他办法让 C# 编译器生成这些 IL 指令，如此一来，在值类型上调用接口方法总是发生装箱。”**

这句话的意思是：

- **只有泛型 + 接口约束** 这个条件组合
- 才能触发 `constrained` 指令

- 其他任何写法（接口变量、非泛型参数）都**不可能**避免装箱

反例：即使是显式接口实现

```
int x = 5;
((IComparable)x).CompareTo(10); // 💥 装箱
```

哪怕 `CompareTo` 是 EIMI，只要**通过接口变量调用**，就必须装箱。

七、原文最后的例外（非常重要）

> **“一个例外是如果值类型实现了一个接口方法，在值类型的实例上调用这个方法不会造成值类型的实例装箱。”**

这句话说的是**直接调用**，不是接口调用：

```
int x = 5;
x.CompareTo(10); // ✅ 无装箱
```

原因我们之前讲过：

- 编译期绑定
- 调用的是 `Int32.CompareTo(int)`
- 不涉及接口调度
- 不需要类型对象指针

⚠️ 但注意：一旦你写成：

```
IComparable<int> t = x;
t.CompareTo(10); // 💥 装箱
```

哪怕是同一个方法，只要**通过接口变量**，装箱就会发生。

八、把整段话压缩成一张决策表

调用方式	是否装箱	原因
`x.CompareTo(10)`	❌	直接调用，编译期绑定
`M<T>(x)` + `where T : IComparable<int>`	❌	`constrained` 指令
`M(IComparable<int> t)` + `t.CompareTo(10)`	✅	接口变量必须指向堆
`((IComparable)x).CompareTo(10)`	✅	值类型转接口必装箱
EIMI 通过接口调用	✅	接口调度需要类型对象指针

九、为什么这个优化这么重要？

考虑 `List<int>.Sort()`：

- 内部调用 `IComparable<int>.CompareTo`
- 如果每次比较都装箱 → 性能灾难
- 有了 `constrained` → **零装箱排序**

这也是为什么 .NET 的集合框架在值类型场景下依然高效。

十、一句话终极总结

> **接口约束让 C# 编译器有机会生成 `constrained` IL 指令，这条指令允许 CLR 在值类型不装箱的前提下完成接口方法调用；没有这个约束，任何接口调用都会导致装箱。**

如果你愿意，下一步我可以：

1. 把 `constrained` 在 ECMA-335 规范里的官方行为画成流程图
2. 或者用 `List<T>.Sort` 的源码带你走一遍“零装箱比较”的完整路径

你选哪个？😊

（注：文档部分内容可能由 AI 生成）